

RGB LED Effects

RGB LED Effects

- 1. WS2812 Overview
- 2. Core Concepts
 - 2.1 WS2812 Communication Protocol
 - 2.2 RMT Peripheral — Protocol Simulator
 - 2.3 led_strip Component Introduction
 - 2.4 Rainbow 7-Color Definition
- 3. Project Architecture and Flow
- 4. Hardware Dependencies
 - 4.1 Method 1: Open Terminal to Download Official Library:
 - 4.2 Method 2: Open idf_component.yml File in main Folder:
- 5. Source Code Analysis
 - 5.1 Color Data Structure
 - 5.2 led_strip Initialization
 - 5.3 Main Loop — Color Switching
- 6. Operation Flow
 - 6.1 Hardware Preparation
 - 6.2 Build and Flash
 - 6.3 Normal Output
- 7. Common Issues

1. WS2812 Overview

WS2812 is an **addressable smart LED** with integrated control chip and RGB LED. Through a single data line, hundreds of LEDs can be cascade-controlled. Each LED has independent RGB color value (0~255 per channel), supporting 16.77 million colors. Widely used in light strips, ambiance lights, pixel screens, and other scenarios.

This experiment uses **RMT (Remote Control Transceiver)** peripheral to simulate WS2812 communication protocol, driving one WS2812 to cycle through 7 rainbow colors.

WS2812 Characteristics:

Feature	Description
Communication Interface	Single wire (no clock line), asynchronous self-synchronization
Data Format	24-bit color (G+R+B each 8-bit, GRB order)
Cascade Capability	Infinite cascade (each LED forwards data, ≤1024 typical)
Refresh Rate	~30Hz (1024 LEDs) ~2kHz (single LED)
Working Voltage	3.5~5.3V (data signal recommended ≥3.3V)

Typical Application Scenarios:

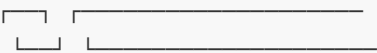
Application Type	Example
Consumer electronics	Ambiance lights, gaming peripherals RGB, chassis lighting effects
Industrial indication	Equipment status lights (green=normal, red=fault, yellow=standby)
Commercial display	LED strips, pixel matrix screens, advertisement light boxes
Smart home	Panel backlight, scene lighting, notification lights

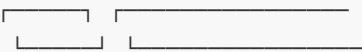
2. Core Concepts

2.1 WS2812 Communication Protocol

WS2812 uses **Non-Return-to-Zero (NRZ)** encoding, using different duty cycles to represent 0 and 1:

1-bit encoding:

"0" code:  (high ~0.4µs, low ~0.85µs)

"1" code:  (high ~0.8µs, low ~0.45µs)

RESET code:  (low >50µs, indicates frame end)

Data Frame Structure (single LED):

[G7..G0] [R7..R0] [B7..B0] → RESET (≥50µs low level)

|— 24-bit GRB data —|

Send order: MSB first, each byte high bit first

Each bit takes ~1.25µs, 24-bit totals ~30µs

WS2812 is **GRB order, not RGB**. Chip datasheet specifies native color order as Green → Red → Blue. Filling RGB values will cause color disorder.

2.2 RMT Peripheral — Protocol Simulator

RMT was originally for infrared remote control, but it's essentially a **programmable digital waveform generator**, precisely controlling duration of each high/low pulse. WS2812's 1-bit encoding can be perfectly represented by RMT's pulse sequence:

RMT configuration:

Resolution: 10MHz → 1 tick = 0.1µs

ws2812 "0": high 4 tick (0.4µs), low 8 tick (0.85µs)

ws2812 "1": high 8 tick (0.8µs), low 4 tick (0.45µs)

Why use RMT instead of SPI or bit-bang?

Solution	Advantage	Disadvantage
RMT (this experiment)	Hardware auto-generates precise timing, no CPU waiting	Occupies one RMT channel (8 available)
SPI	Faster speed	Needs MOSI pin + WS2812 protocol adaptation
Bit-bang (GPIO)	Any GPIO, no resource limit	Occupies CPU, timing jitter large, not recommended

2.3 led_strip Component Introduction

ESP-IDF Component Manager provides `espressif/led_strip` component, encapsulating WS2812's underlying RMT/SPI operations:

API	Function
<code>led_strip_new_rmt_device()</code>	Create RMT-driven WS2812 instance
<code>led_strip_set_pixel()</code>	Set RGB value for specified pixel
<code>led_strip_refresh()</code>	Refresh display (send data to LED)
<code>led_strip_clear()</code>	Clear all pixels (all off)

`led_strip` component is introduced through IDF Component Manager (`idf_component.yml`), automatically downloaded during compilation.

2.4 Rainbow 7-Color Definition

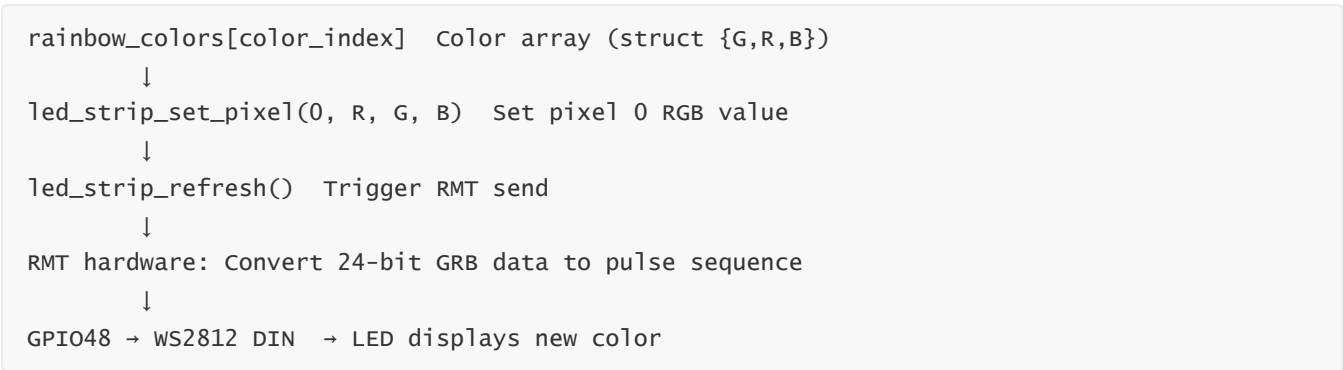
Color	G	R	B	visual effect
Red	0	150	0	Pure red
Orange	60	150	0	Red+green → orange
Yellow	150	150	0	Red+green full → yellow
Green	150	0	0	Pure green
Cyan	150	0	150	Green+blue → cyan
Blue	0	0	150	Pure blue
Purple	0	120	150	Red+blue → purple

Brightness set to 150 instead of 255 because WS2812 at 255 full brightness has high power consumption and is glaring. 150 at ~59% brightness is more comfortable for experiment environment while reducing power supply ripple.

3. Project Architecture and Flow

```
app_main()
├─ led_strip_new_rmt_device()    Create RMT WS2812 device
│   strip_config: GPIO48, 1 LED, WS2812, GRB format
│   rmt_config:    10MHz resolution, DMA disabled
│
├─ led_strip_clear()            All off initial state
│
└─ while(1) Main Loop
    ├─ printf("Current color: %s")    Print current color name
    ├─ led_strip_set_pixel(0, R, G, B) Set pixel 0 RGB
    ├─ led_strip_refresh()            Refresh display (send data)
    ├─ color_index++                  Switch to next color
    └─ vTaskDelay(1000ms)             Color switch interval
```

Data Flow:



4. Hardware Dependencies

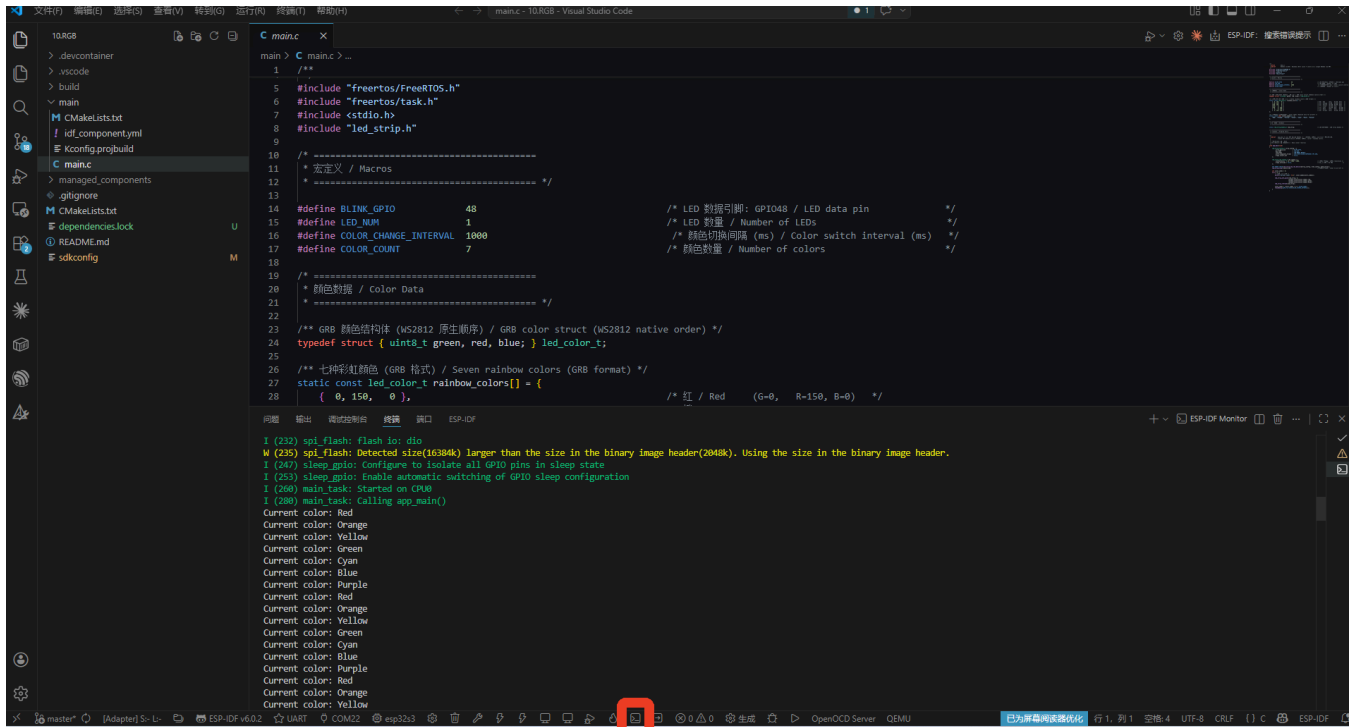
Hardware Connection:

Pin	Peripheral	Description
GPIO48	WS2812 data line (DIN)	RMT pulse output
VDD (5V/3.3V)	WS2812 VCC	Power (3.3V can light up but dimmer)
GND	Common ground	ESP32-S3 GND ↔ WS2812 GND

Dependencies: `freertos`, `led_strip` (espressif/led_strip via IDF Component Manager)

4.1 Method 1: Open Terminal to Download Official Library:

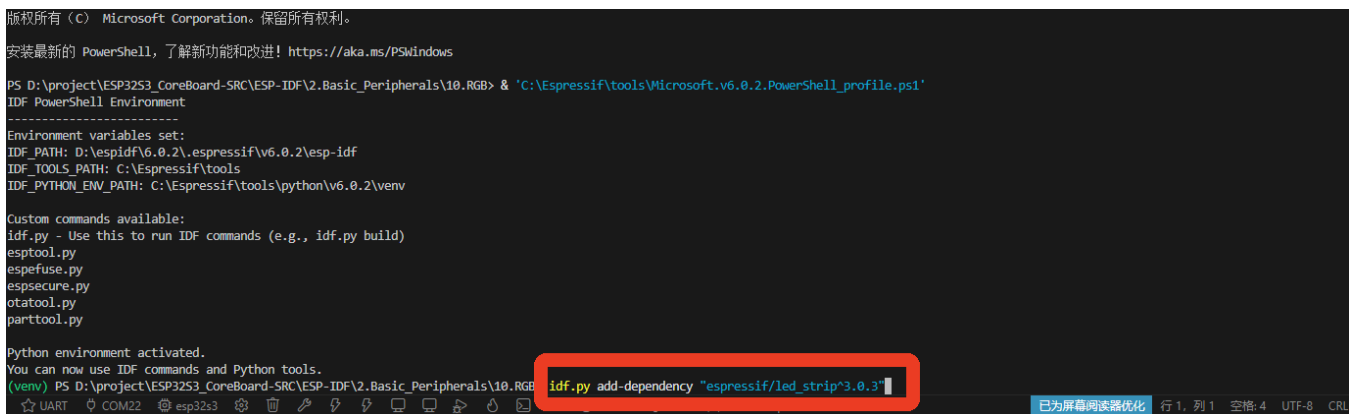
- 1. Open ESP-IDF terminal



```
10.RGB
> .devcontainer
> .vscode
> build
> main
  CMakelists.txt
  idf_component.yml
  Kconfig.projbuild
  C main.c
> managed_components
  .gitignore
  CMakelists.txt
  dependencies.lock
  README.md
  sdkconfig

main > C main.c > ...
1  /**
2
3  5 #include "freertos/freertos.h"
4 #include "freertos/task.h"
5 #include <stdio.h>
6 #include "led_strip.h"
7
8  9
9  10 /* =====
11  11 * 宏定义 / Macros
12  12 * ===== */
13
14 #define BLINK_GPIO 48 /* LED 引脚: GPIO48 / LED data pin */
15 #define LED_NUM 1 /* LED 数量 / Number of LEDs */
16 #define COLOR_CHANGE_INTERVAL 1000 /* 颜色切换间隔 (ms) / color switch interval (ms) */
17 #define COLOR_COUNT 7 /* 颜色数量 / Number of colors */
18
19  19 /* =====
20  20 * 颜色数据 / Color Data
21  21 * ===== */
22
23  23 /** GRB 颜色结构体 (WS2812 原生顺序) / GRB color struct (WS2812 native order) */
24  24 typedef struct { uint8_t green, red, blue; } led_color_t;
25  25
26  26 /** 七种彩虹颜色 (GRB 格式) / Seven rainbow colors (GRB format) */
27  27 static const led_color_t rainbow_colors[] = {
28  28     { 0, 150, 0 }, /* 红 / Red (G=0, R=150, B=0) */
29  29
30  30
31  31
32  32
33  33
34  34
35  35
36  36
37  37
38  38
39  39
40  40
41  41
42  42
43  43
44  44
45  45
46  46
47  47
48  48
49  49
50  50
51  51
52  52
53  53
54  54
55  55
56  56
57  57
58  58
59  59
60  60
61  61
62  62
63  63
64  64
65  65
66  66
67  67
68  68
69  69
70  70
71  71
72  72
73  73
74  74
75  75
76  76
77  77
78  78
79  79
80  80
81  81
82  82
83  83
84  84
85  85
86  86
87  87
88  88
89  89
90  90
91  91
92  92
93  93
94  94
95  95
96  96
97  97
98  98
99  99
100 100
101 101
102 102
103 103
104 104
105 105
106 106
107 107
108 108
109 109
110 110
111 111
112 112
113 113
114 114
115 115
116 116
117 117
118 118
119 119
120 120
121 121
122 122
123 123
124 124
125 125
126 126
127 127
128 128
129 129
130 130
131 131
132 132
133 133
134 134
135 135
136 136
137 137
138 138
139 139
140 140
141 141
142 142
143 143
144 144
145 145
146 146
147 147
148 148
149 149
150 150
151 151
152 152
153 153
154 154
155 155
156 156
157 157
158 158
159 159
160 160
161 161
162 162
163 163
164 164
165 165
166 166
167 167
168 168
169 169
170 170
171 171
172 172
173 173
174 174
175 175
176 176
177 177
178 178
179 179
180 180
181 181
182 182
183 183
184 184
185 185
186 186
187 187
188 188
189 189
190 190
191 191
192 192
193 193
194 194
195 195
196 196
197 197
198 198
199 199
200 200
201 201
202 202
203 203
204 204
205 205
206 206
207 207
208 208
209 209
210 210
211 211
212 212
213 213
214 214
215 215
216 216
217 217
218 218
219 219
220 220
221 221
222 222
223 223
224 224
225 225
226 226
227 227
228 228
229 229
230 230
231 231
232 232
233 233
234 234
235 235
236 236
237 237
238 238
239 239
240 240
241 241
242 242
243 243
244 244
245 245
246 246
247 247
248 248
249 249
250 250
251 251
252 252
253 253
254 254
255 255
256 256
257 257
258 258
259 259
260 260
261 261
262 262
263 263
264 264
265 265
266 266
267 267
268 268
269 269
270 270
271 271
272 272
273 273
274 274
275 275
276 276
277 277
278 278
279 279
280 280
281 281
282 282
283 283
284 284
285 285
286 286
287 287
288 288
289 289
290 290
291 291
292 292
293 293
294 294
295 295
296 296
297 297
298 298
299 299
300 300
301 301
302 302
303 303
304 304
305 305
306 306
307 307
308 308
309 309
310 310
311 311
312 312
313 313
314 314
315 315
316 316
317 317
318 318
319 319
320 320
321 321
322 322
323 323
324 324
325 325
326 326
327 327
328 328
329 329
330 330
331 331
332 332
333 333
334 334
335 335
336 336
337 337
338 338
339 339
340 340
341 341
342 342
343 343
344 344
345 345
346 346
347 347
348 348
349 349
350 350
351 351
352 352
353 353
354 354
355 355
356 356
357 357
358 358
359 359
360 360
361 361
362 362
363 363
364 364
365 365
366 366
367 367
368 368
369 369
370 370
371 371
372 372
373 373
374 374
375 375
376 376
377 377
378 378
379 379
380 380
381 381
382 382
383 383
384 384
385 385
386 386
387 387
388 388
389 389
390 390
391 391
392 392
393 393
394 394
395 395
396 396
397 397
398 398
399 399
400 400
401 401
402 402
403 403
404 404
405 405
406 406
407 407
408 408
409 409
410 410
411 411
412 412
413 413
414 414
415 415
416 416
417 417
418 418
419 419
420 420
421 421
422 422
423 423
424 424
425 425
426 426
427 427
428 428
429 429
430 430
431 431
432 432
433 433
434 434
435 435
436 436
437 437
438 438
439 439
440 440
441 441
442 442
443 443
444 444
445 445
446 446
447 447
448 448
449 449
450 450
451 451
452 452
453 453
454 454
455 455
456 456
457 457
458 458
459 459
460 460
461 461
462 462
463 463
464 464
465 465
466 466
467 467
468 468
469 469
470 470
471 471
472 472
473 473
474 474
475 475
476 476
477 477
478 478
479 479
480 480
481 481
482 482
483 483
484 484
485 485
486 486
487 487
488 488
489 489
490 490
491 491
492 492
493 493
494 494
495 495
496 496
497 497
498 498
499 499
500 500
501 501
502 502
503 503
504 504
505 505
506 506
507 507
508 508
509 509
510 510
511 511
512 512
513 513
514 514
515 515
516 516
517 517
518 518
519 519
520 520
521 521
522 522
523 523
524 524
525 525
526 526
527 527
528 528
529 529
530 530
531 531
532 532
533 533
534 534
535 535
536 536
537 537
538 538
539 539
540 540
541 541
542 542
543 543
544 544
545 545
546 546
547 547
548 548
549 549
550 550
551 551
552 552
553 553
554 554
555 555
556 556
557 557
558 558
559 559
560 560
561 561
562 562
563 563
564 564
565 565
566 566
567 567
568 568
569 569
570 570
571 571
572 572
573 573
574 574
575 575
576 576
577 577
578 578
579 579
580 580
581 581
582 582
583 583
584 584
585 585
586 586
587 587
588 588
589 589
590 590
591 591
592 592
593 593
594 594
595 595
596 596
597 597
598 598
599 599
600 600
601 601
602 602
603 603
604 604
605 605
606 606
607 607
608 608
609 609
610 610
611 611
612 612
613 613
614 614
615 615
616 616
617 617
618 618
619 619
620 620
621 621
622 622
623 623
624 624
625 625
626 626
627 627
628 628
629 629
630 630
631 631
632 632
633 633
634 634
635 635
636 636
637 637
638 638
639 639
640 640
641 641
642 642
643 643
644 644
645 645
646 646
647 647
648 648
649 649
650 650
651 651
652 652
653 653
654 654
655 655
656 656
657 657
658 658
659 659
660 660
661 661
662 662
663 663
664 664
665 665
666 666
667 667
668 668
669 669
670 670
671 671
672 672
673 673
674 674
675 675
676 676
677 677
678 678
679 679
680 680
681 681
682 682
683 683
684 684
685 685
686 686
687 687
688 688
689 689
690 690
691 691
692 692
693 693
694 694
695 695
696 696
697 697
698 698
699 699
700 700
701 701
702 702
703 703
704 704
705 705
706 706
707 707
708 708
709 709
710 710
711 711
712 712
713 713
714 714
715 715
716 716
717 717
718 718
719 719
720 720
721 721
722 722
723 723
724 724
725 725
726 726
727 727
728 728
729 729
730 730
731 731
732 732
733 733
734 734
735 735
736 736
737 737
738 738
739 739
740 740
741 741
742 742
743 743
744 744
745 745
746 746
747 747
748 748
749 749
750 750
751 751
752 752
753 753
754 754
755 755
756 756
757 757
758 758
759 759
760 760
761 761
762 762
763 763
764 764
765 765
766 766
767 767
768 768
769 769
770 770
771 771
772 772
773 773
774 774
775 775
776 776
777 777
778 778
779 779
780 780
781 781
782 782
783 783
784 784
785 785
786 786
787 787
788 788
789 789
790 790
791 791
792 792
793 793
794 794
795 795
796 796
797 797
798 798
799 799
800 800
801 801
802 802
803 803
804 804
805 805
806 806
807 807
808 808
809 809
810 810
811 811
812 812
813 813
814 814
815 815
816 816
817 817
818 818
819 819
820 820
821 821
822 822
823 823
824 824
825 825
826 826
827 827
828 828
829 829
830 830
831 831
832 832
833 833
834 834
835 835
836 836
837 837
838 838
839 839
840 840
841 841
842 842
843 843
844 844
845 845
846 846
847 847
848 848
849 849
850 850
851 851
852 852
853 853
854 854
855 855
856 856
857 857
858 858
859 859
860 860
861 861
862 862
863 863
864 864
865 865
866 866
867 867
868 868
869 869
870 870
871 871
872 872
873 873
874 874
875 875
876 876
877 877
878 878
879 879
880 880
881 881
882 882
883 883
884 884
885 885
886 886
887 887
888 888
889 889
890 890
891 891
892 892
893 893
894 894
895 895
896 896
897 897
898 898
899 899
900 900
901 901
902 902
903 903
904 904
905 905
906 906
907 907
908 908
909 909
910 910
911 911
912 912
913 913
914 914
915 915
916 916
917 917
918 918
919 919
920 920
921 921
922 922
923 923
924 924
925 925
926 926
927 927
928 928
929 929
930 930
931 931
932 932
933 933
934 934
935 935
936 936
937 937
938 938
939 939
940 940
941 941
942 942
943 943
944 944
945 945
946 946
947 947
948 948
949 949
950 950
951 951
952 952
953 953
954 954
955 955
956 956
957 957
958 958
959 959
960 960
961 961
962 962
963 963
964 964
965 965
966 966
967 967
968 968
969 969
970 970
971 971
972 972
973 973
974 974
975 975
976 976
977 977
978 978
979 979
980 980
981 981
982 982
983 983
984 984
985 985
986 986
987 987
988 988
989 989
990 990
991 991
992 992
993 993
994 994
995 995
996 996
997 997
998 998
999 999
1000 1000
```

2. Enter download command in input box



```
版权所有 (C) Microsoft Corporation。保留所有权利。

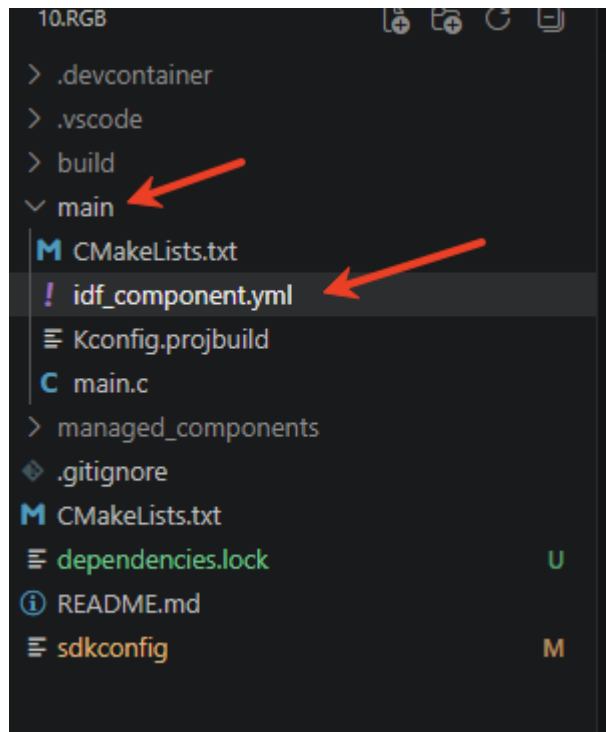
安装最新的 PowerShell，了解新功能和改进！https://aka.ms/PSWindows

PS D:\project\ESP32S3_CoreBoard-SRC\ESP-IDF\2.Basic_Peripherals\10.RGB> & 'C:\Espressif\tools\Microsoft.v6.0.2.PowerShell_profile.ps1'
IDF PowerShell Environment
-----
Environment variables set:
IDF_PATH: D:\espidf\6.0.2\espressif\v6.0.2\esp-idf
IDF_TOOLS_PATH: C:\Espressif\tools
IDF_PYTHON_ENV_PATH: C:\Espressif\tools\python\v6.0.2\venv

Custom commands available:
idf.py - Use this to run IDF commands (e.g., idf.py build)
esptool.py
espfuse.py
espssecure.py
otatool.py
parttool.py

Python environment activated.
You can now use IDF commands and Python tools.
(venv) PS D:\project\ESP32S3_CoreBoard-SRC\ESP-IDF\2.Basic_Peripherals\10.RGB> idf.py add-dependency "espressif/led_strip@3.0.3"
```

4.2 Method 2: Open idf_component.yml File in main Folder:



```
! idf_component.yml X
main > ! idf_component.yml
1  ## IDF Component Manager Manifest File
2  dependencies:
3    ## Required IDF version
4    idf:
5      version: '>=4.1.0'
6    ## Put list of dependencies here
7    ## For components maintained by Espressif:
8    ## component: "~1.0.0"
9    ## For 3rd party components:
10   ## username/component: ">=1.0.0,<2.0.0"
11   ## username2/component2:
12   ##   version: "~1.0.0"
13   ## For transient dependencies `public` flag can be set.
14   ## `public` flag doesn't have an effect dependencies of the `main` component.
15   ## All dependencies of `main` are public by default.
16   ...
17   espressif/led_strip: ^3.0.3
18
```

Add led_strip library: Execute the following command in project directory, IDF Component Manager will automatically download and manage dependency.

```
idf.py add-dependency "espressif/led_strip^3.0.3"
```

5. Source Code Analysis

5.1 Color Data Structure

First define basic constants—which IO LED is connected to, how many LEDs, how many colors to cycle. Then store each color's RGB value in array, because WS2812's data order is GRB (green→red→blue), so green is written first in struct. Finally configure a string array of color names for serial printing.

```
#define BLINK_GPIO          48
#define LED_NUM             1
// Number of colors: 7
#define COLOR_COUNT         7
typedef struct {
    uint8_t green, red, blue;
} led_color_t;

// Seven rainbow colors (GRB format)
static const led_color_t rainbow_colors[] = {
    { 0, 150, 0 }, // Red    (G=0, R=150, B=0)
    { 60, 150, 0 }, // Orange (G=60, R=150, B=0)
    { 150, 150, 0 }, // Yellow (G=150, R=150, B=0)
    { 150, 0, 0 }, // Green  (G=150, R=0, B=0)
    { 150, 0, 150 }, // Cyan   (G=150, R=0, B=150)
    { 0, 0, 150 }, // Blue   (G=0, R=0, B=150)
    { 0, 120, 150 }, // Purple (G=0, R=120, B=150)
};

// Color names (printf, English only)
static const char *color_names[] = {
    "Red", "Orange", "Yellow", "Green", "Cyan", "Blue", "Purple"
};
```

Struct member order is green, red, blue: Matches WS2812's native GRB order. When code calls `led_strip_set_pixel(0, color.red, color.green, color.blue)`, the RGB order passed is handled internally by `led_strip` because `LED_STRIP_COLOR_COMPONENT_FMT_GRB` is configured.

5.2 led_strip Initialization

Initialization in two steps: first configure LED parameters (pin, number, model, color format), second configure RMT parameters (clock resolution, whether to use DMA). Then call `led_strip_new_rmt_device()` to create device handle, then `led_strip_clear()` to turn all LEDs off, ensuring powered-on state is deterministic.

```
void app_main(void)
{
    led_strip_config_t strip_config = {
        .strip_gpio_num    = BLINK_GPIO, // GPIO48
        .max_leds           = LED_NUM,    // 1 LED
        .led_model          = LED_MODEL_WS2812,
        .color_component_format = LED_STRIP_COLOR_COMPONENT_FMT_GRB,
        // Non-inverted output
        .flags.invert_out   = false,
    };
```

```

};
led_strip_rmt_config_t rmt_config = {
    // 10MHz = 0.1µs/tick (100ns precision)
    .resolution_hz = 10 * 1000 * 1000,
    // No DMA (only 1 LED)
    .flags.with_dma = false,
};

ESP_ERROR_CHECK(led_strip_new_rmt_device(&strip_config, &rmt_config, &led_strip));
// Initial all off
led_strip_clear(led_strip);

```

Why 10MHz? WS2812 timing precision requirement is within $\pm 150\text{ns}$. 10MHz (100ns/tick) provides sufficient precision, while not having frequency too high causing RMT memory (64 entries/channel) insufficient to store one frame of data.

Why `with_dma = false`? Only 1 LED needs 24 RMT entries, far less than RMT memory limit (64/channel). Enabling DMA increases configuration complexity. For controlling hundreds of LEDs, DMA can bypass RMT memory limit.

5.3 Main Loop — Color Switching

After initialization, enter infinite loop: first use `printf` to print current color name for easy monitoring in serial monitor. Then call `led_strip_set_pixel()` to fill current color's RGB value, then `led_strip_refresh()` to send data through RMT, LED changes color. Finally index +1 with modulo for cycling, `vTaskDelay` 1 second delay before switching to next color.

```

int color_index = 0;
while (1) {
    // Print current color
    printf("Current color: %s\n", color_names[color_index]);

    // Pixel index 0 (first LED)
    led_strip_set_pixel(led_strip, 0,
                        rainbow_colors[color_index].red,
                        rainbow_colors[color_index].green,
                        rainbow_colors[color_index].blue);

    // Send data to LED
    led_strip_refresh(led_strip);

    // Cycle index
    color_index = (color_index + 1) % COLOR_COUNT;
    // Delay 1s
    vTaskDelay(pdMS_TO_TICKS(COLOR_CHANGE_INTERVAL));
}
}

```

`led_strip_set_pixel()` parameter order is (R, G, B), different from struct's {G, R, B} definition—because led_strip API uniformly uses (handle, index, R, G, B) interface, color component sorting is handled by `color_component_format` configuration.

`printf()` instead of `ESP_LOGI()`: GPIO48 has no conflict with UART0 (unlike GPIO43/44), serial output can be used. Using `printf` to output color name (English only, following project encoding standard), convenient for debugging and confirming current color.

6. Operation Flow

6.1 Hardware Preparation

1. WS2812 module (or first LED of light strip)
2. Wiring:

ESP32-S3		WS2812
GPIO48	→	DIN (data input)
VDD (5V)	→	VCC (5V brighter)
GND	→	GND (common ground)

5V power supply recommended: 3.3V can light WS2812, but LED color is dim and unstable (WS2812 datasheet requires $V_{IH} \geq 0.7 \times V_{DD}$, at 3.3V supply input high threshold $\sim 2.3V$, GPIO's 3.3V output is above that threshold).

6.2 Build and Flash

For detailed flashing process, see: [1. Before Development](#) → [3. Build and Flash](#)

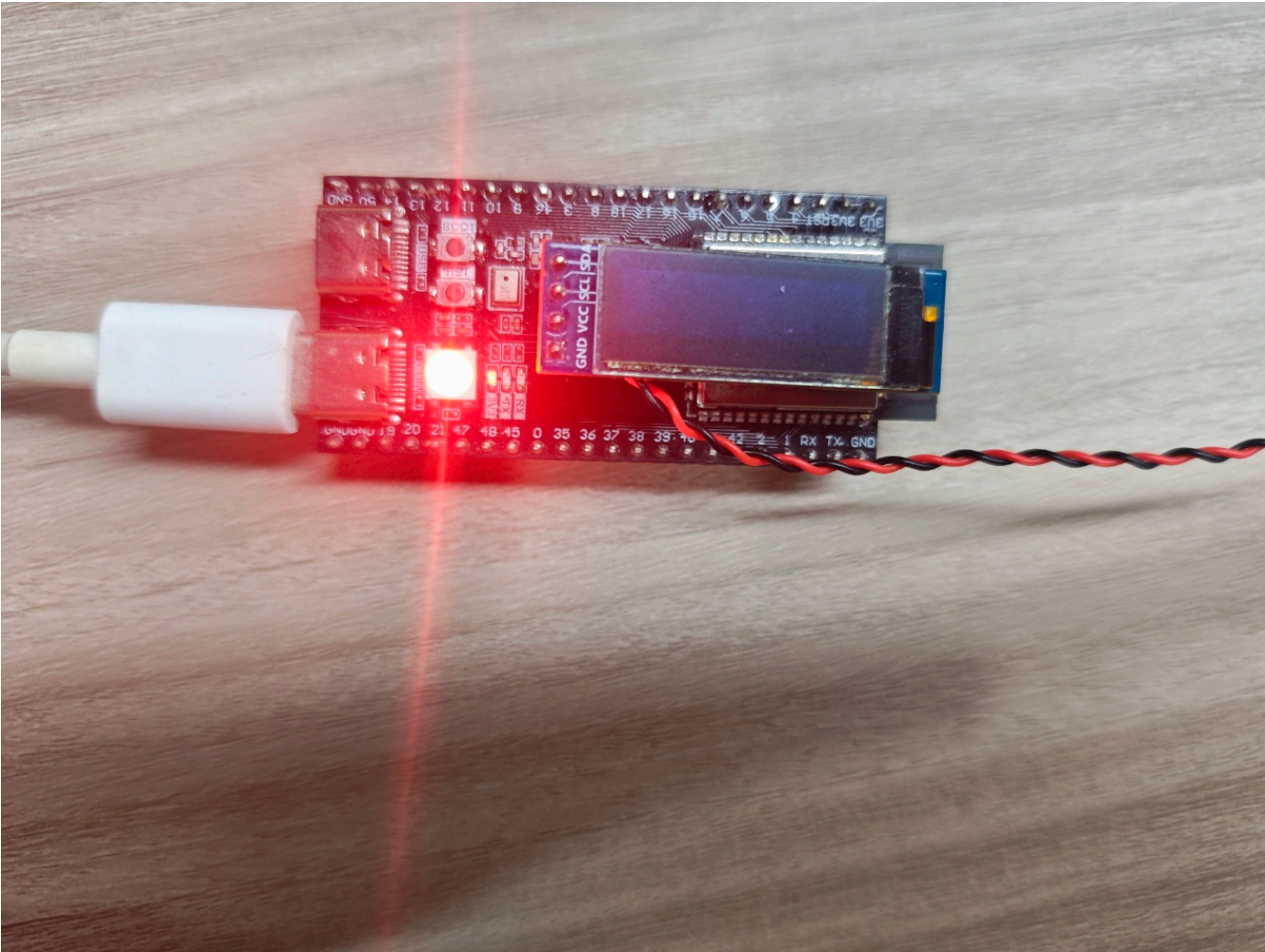
6.3 Normal Output

Serial monitor:

```
Current color: Red
Current color: Orange
Current color: Yellow
Current color: Green
Current color: Cyan
Current color: Blue
Current color: Purple
Current color: Red      ← Cycle
...
```

```
I (260) main_task: Started on CPU0
I (280) main_task: Calling app_main()
Current color: Red
Current color: Orange
Current color: Yellow
Current color: Green
Current color: Cyan
Current color: Blue
Current color: Purple
Current color: Red
Current color: Orange
Current color: Yellow
Current color: Green
Current color: Cyan
```

LED phenomenon: Switches one color every 1 second, in order red→orange→yellow→green→cyan→blue→purple→red cycle.



7. Common Issues

Phenomenon	Cause	Solution
LED doesn't light up	DIN connected wrong / wrong GPIO / insufficient power	Check wiring, confirm GPIO48; use multimeter to measure VDD voltage

Phenomenon	Cause	Solution
Wrong color (red becomes green)	RGB/GRB order configuration error	Confirm <code>color_component_format = LED_STRIP_COLOR_COMPONENT_FMT_GRB</code>
LED flickers/random color jumps	Timing imprecise / large power supply ripple	Add 100μF electrolytic capacitor + 0.1μF ceramic capacitor in parallel between VDD-GND
When cascading multiple LEDs, first normal, later ones show wrong colors	Later LED data input disturbed / power voltage drop	Solder decoupling capacitor every N LEDs; shorten data line length
<code>led_strip_new_rmt_device</code> fails	RMT channel resources exhausted	Check if other code occupies RMT channel
Doesn't light after first flash	LED state not initialized	<code>led_strip_clear</code> ensures RESET pulse is sent
Colors too dim	3.3V power supply / RGB values too low	Use 5V power supply; increase RGB values (e.g., 255)